

# # PROJECT REPORT

## ## Structured Tool Representation and Trajectory Supervision for Multi-Turn Tool-Use Learning

### ## Abstract

This undergraduate research project studies whether structured tool representations can improve learned multi-turn tool-use consistency in smaller language models. The work focuses on data-centric supervision quality rather than building a production coding system. A preprocessing pipeline converts noisy interaction logs into normalized and compressed trajectories, followed by QLoRA fine-tuning of `Qwen2.5-Coder-3B-Instruct`. The resulting model outputs are analyzed through structured JSON generation probes, where both successful and failed artifacts are observed. The repository demonstrates a complete prototype workflow for investigating representation quality, trajectory structuring, and tool-call consistency.

### ## 1. Introduction

Many tool-use training sources contain inconsistencies in tool naming, low-signal logs, repeated steps, and weak multi-turn continuity. These issues reduce the quality of supervision signals, especially for smaller models.

This project investigates a representation-centered alternative: improve the supervision substrate itself through normalization, filtering, compression, and structured formatting. The main hypothesis is that better trajectory representation can improve learned structured tool-use behavior.

### ## 2. Problem Statement

The project addresses four practical problems in raw tool-use data:

1. Tool semantic fragmentation (multiple names for similar actions).
2. Large noise-to-signal ratio in logs.
3. Repetitive and low-value trajectory steps.
4. Weak structural consistency in generated outputs after training.

The goal is not to claim full autonomy, but to test whether structured supervision improves consistency of tool invocation patterns.

### ## 3. Domain Analysis Alignment

From `domain\_research.pdf`, key observations are:

1. Prompt-level format compliance alone does not guarantee tool-semantic consistency.
2. Synthetic trajectories may look structured but lack realistic iterative depth.
3. SWE-chat style real interactions contain richer iterative tool-use patterns.
4. Preprocessing quality is central when converting raw logs into trainable signals.

This report follows that framing directly.

### ## 4. Objectives

1. Normalize tool names into a compact canonical schema.
2. Preserve useful behavioral signal while reducing noise and redundancy.
3. Build structured trajectory supervision (`dataset\_v8.jsonl`).
4. Fine-tune a 3B model via QLoRA.
5. Probe post-training structural consistency using JSON output validation.

### ## 5. Methodology

### ### 5.1 Preprocessing and Trajectory Structuring (`gen12.py`)

Implemented stages:

1. Load local dataset from `./swe\_chat\_local`.
2. Reconstruct sessions (`session\_id`, `turn\_number`).
3. Extract tool-use and tool-result steps.
4. Canonicalize tool semantics into:
  - `read\_files`
  - `search\_files`
  - `modify\_files`
  - `write\_files`
  - `run\_command`
5. Normalize paths to canonical workspace form.
6. Clean and compress output fields.
7. Deduplicate repeated reads/commands.
8. Filter sessions via structural heuristics and scoring.
9. Reconstruct compact goal/meta fields.
10. Save to `dataset\_v8.jsonl`.

### ### 5.2 Structured Supervision and QLoRA Training (`train.py`)

1. Base model: `Qwen/Qwen2.5-Coder-3B-Instruct`.
2. Quantization: 4-bit NF4.
3. Adaptation: LoRA on attention and MLP projection modules.
4. Supervision format: chat-style text containing structured JSON trajectory targets.
5. Trainer: TRL `SFTTrainer`.
6. Output artifacts: adapter + tokenizer in `./agent\_lora`.

Training settings used in script:

- epochs: 3
- learning rate: 2e-4
- batch size: 1
- gradient accumulation: 16
- max sequence length: 2048
- lora\_r: 32
- lora\_alpha: 64
- lora\_dropout: 0.05

### ### 5.3 Structured Output Probe (`agentTester.py`)

1. Load quantized base model.
2. Optional adapter loading via `USE\_ADAPTER`.
3. Prompt strict JSON-only schema output.
4. Generate with sampling.
5. Parse first balanced JSON object.
6. Save `output.json` on success.

This stage functions as an output-structure probe rather than a deployment runtime.

## ## 6. Technologies Used

- Python
- PyTorch
- Transformers

- Datasets
- PEFT (LoRA/QLoRA)
- TRL (SFTTrainer)
- BitsAndBytes
- tqdm

### ## 7. Repository Artifacts and What They Show

- `output1.json`: valid structured output sample.
- `outputF.json`: invalid/truncated sample.

#### Interpretation:

- Structured generation can succeed.
- Structural failures still occur.
- Reliability remains an open research direction.

### ## 8. Research Experience and Observations

1. Preprocessing quality had strong practical impact on supervision quality.
2. Tool normalization reduced semantic entropy and improved consistency potential.
3. Compression improved efficiency but introduced detail-loss tradeoffs.
4. Structured supervision helped produce coherent output structure in some runs.
5. Output failure cases remained, motivating stronger validation and decoding control.

These observations are grounded in repository code and artifacts and should be treated as prototype-level findings.

### ## 9. Limitations

1. Evidence in repository is mostly qualitative/structural.
2. No integrated benchmark framework with large-scale metrics is provided.
3. Compression can reduce fine-grained edit semantics.
4. Validation currently checks parseability, not complete formal schema conformance.

### ## 10. Future Scope

1. Add strict schema validator and constrained decoding loop.
2. Add quantitative consistency evaluation (validity + tool-sequence fidelity).
3. Improve compressed representations to retain more semantic edit information.
4. Expand trajectory diversity and harder multi-turn settings.
5. Add reproducible experiment tracking and reporting.

### ## 11. Conclusion

This project contributes a data-centric prototype for studying structured tool embeddings in smaller models. The main value lies in representation engineering: normalization, compression, filtering, and structured supervision design. The coding-task JSON outputs are evaluation artifacts that help inspect learned structured tool-use behavior. Overall, the repository provides a realistic undergraduate research foundation for deeper experimentation on tool semantics and trajectory quality.

### ## 12. Team Contribution Note

From `extra.info.txt`:

- Research and dataset pattern: Suhas, Swayam, Nikhil
- Dataset filtering/training/formatting collaboration: Shrikar, Dalip
- Tool-interaction pipeline: Harshit, Rahul